

JAVA INTERVIEW PREPARATION

CHAPTER 72

RECURSION, BACKTRACKING,
AND SEARCH-SPACE PRUNING

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

Recursion, Backtracking, and Search-Space Pruning

Senior / Lead Java Developer Reading Chapter

Recursion expresses a problem in terms of smaller instances of the same problem. Backtracking adds controlled choice: make a decision, explore its consequences, then undo the decision before trying another branch. These techniques are powerful because the code can mirror a tree-shaped search space. They are dangerous when the state boundary is unclear, branching is unbounded, or recursion depth is treated as free.

Senior interviews are rarely about producing a permutation template from memory. They test whether you can define the recursive contract, prove progress and termination, separate path state from global state, prune only when logically safe, estimate exponential growth, and connect the algorithm to production resource limits.

A recursive call is a contract

Every recursive function needs three things:

- a state that describes one smaller subproblem;
- a base case that returns a correct answer without another call;
- a transition that makes measurable progress toward the base case.

```
static long factorial(int n) {  
    if (n < 0) throw new IllegalArgumentException("n");  
    if (n <= 1) return 1;  
    return Math.multiplyExact(n, factorial(n - 1));  
}
```

The contract is: return $n!$ for a nonnegative n , or fail on overflow. The progress measure is n , which decreases toward 1. Saying only "the function calls itself" misses the correctness structure.

```
factorial(4)  
  waits for factorial(3)  
    waits for factorial(2)  
      waits for factorial(1) = 1  
    returns 2  
  returns 6  
returns 24
```

Each suspended call retains parameters, local variables, and a return location in its stack frame. Time may be linear while stack space is also linear. Java does not guarantee tail-call elimination, so a tail-recursive method can still overflow the stack.

Recursive state versus accumulated state

Some functions return the subproblem answer directly. Others carry an accumulator or construct an output. Be precise about who owns mutable state.

```
static int sum(int[] values, int index) {  
    if (index == values.length) return 0;  
    return values[index] + sum(values, index + 1);  
}
```

This is easy to reason about but uses $O(n)$ stack. An iterative loop is safer and usually clearer for a simple linear fold. Recursion is most valuable when the input is recursive, such as a tree, or when the algorithm

explores choices.

Hidden global accumulators make repeated calls and concurrency unsafe:

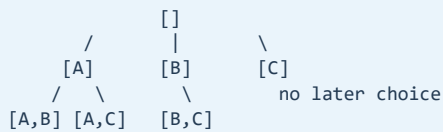
```
// Avoid shared result fields for request-scoped search state.  
private final List<List<Integer>> results = new ArrayList<>();
```

Prefer method-local state passed explicitly. If results are enormous, stream them through a callback or iterator-like search rather than collecting everything.

Backtracking is depth-first search over decisions

A backtracking algorithm maintains a partial candidate. At each decision point it chooses one option, extends the candidate, explores, and restores the previous state.

choices {A, B, C}, choose two



The canonical shape is:

```
void search(State state) {  
    if (complete(state)) {  
        record(state);  
        return;  
    }  
    for (Choice choice : candidates(state)) {  
        if (!valid(state, choice)) continue;  
        apply(state, choice);  
        search(state);  
        undo(state, choice);  
    }  
}
```

The important line is `undo`. The next sibling branch must see the same state that existed before the choice. Copying state per branch avoids undo bugs but increases allocation. Mutate-and-restore is efficient when the invariant is disciplined.

If exploration can throw, timeout, or be cancelled through an exception, restoration should be exception-safe:

```
apply(state, choice);  
try {  
    search(state);  
} finally {  
    undo(state, choice);  
}
```

This matters when the state belongs to a reusable solver object or when a caller catches the exception and inspects it. A purely method-local state that is abandoned on failure may not require cleanup for correctness, but resource handles and externally visible mutations still do.

Combinations avoid duplicate orderings

For combinations, carry a start index so later calls only choose later positions.

```

static List<List<Integer>> combinations(int[] values, int k) {
    if (k < 0 || k > values.length) return List.of();
    List<List<Integer>> output = new ArrayList<>();
    choose(values, k, 0, new ArrayList<>(), output);
    return output;
}

private static void choose(int[] values, int k, int start,
                           List<Integer> path,
                           List<List<Integer>> output) {
    if (path.size() == k) {
        output.add(List.copyOf(path));
        return;
    }

    int needed = k - path.size();
    for (int i = start; i <= values.length - needed; i++) {
        path.add(values[i]);
        choose(values, k, i + 1, path, output);
        path.remove(path.size() - 1);
    }
}

```

The loop bound is a pruning rule. If `needed` items remain, starting after `values.length - needed` cannot complete a solution. The recorded path must be copied; storing the mutable `path` reference would make every result reflect later mutations.

Combinations have output size " n choose k ." No algorithm can materialize them in less space than the output itself. Complexity statements should include output size rather than calling the recursion merely $O(n)$.

Permutations require used-state or swapping

A permutation chooses every item once at every position. One implementation tracks used indices.

```

static void permute(int[] values, boolean[] used,
                   List<Integer> path, List<List<Integer>> output) {
    if (path.size() == values.length) {
        output.add(List.copyOf(path));
        return;
    }

    for (int i = 0; i < values.length; i++) {
        if (used[i]) continue;
        used[i] = true;
        path.add(values[i]);
        permute(values, used, path, output);
        path.remove(path.size() - 1);
        used[i] = false;
    }
}

```

There are $n!$ outputs for distinct values, each of length n , so materialization is $O(n * n!)$ time and space. The visited array distinguishes positions, not values.

With duplicate values, naive index selection emits duplicate permutations. Sort first, then skip a value when it equals the previous value and the previous equal position has not been used in the current level:

```

if (i > 0 && values[i] == values[i - 1] && !used[i - 1]) continue;

```

The condition is subtle. It removes symmetric sibling choices while still permitting two equal values at different depths.

Subsets form an include/exclude tree

Each item creates two choices: include or exclude.

```
items [A, B]

      []
    exclude A / \ include A
          [] [A]
        B? /\   /\ B?
          [] [B] [A] [A,B]
```

The full search has 2^n leaves. An iterative bit-mask method represents each subset by an n-bit mask when n fits the chosen integer width, but it has the same output explosion. The representation changes constants and control flow, not the number of subsets.

Production APIs should never expose unbounded subset or permutation generation from user-controlled n. Validate size, cap results, support cancellation, and consider lazy generation.

Constraint search: N-Queens

N-Queens places one queen in every row so no two share a column or diagonal. The board is the output representation; the search state can be much smaller.

```
row 0: . Q . .
row 1: . . . Q
row 2: Q . . .
row 3: . . Q .

columns:      col
down diagonal: row - col
up diagonal:  row + col
```

```
static int countQueens(int n) {
    if (n < 0) throw new IllegalArgumentException("n");
    boolean[] columns = new boolean[n];
    boolean[] down = new boolean[Math.max(0, 2 * n - 1)];
    boolean[] up = new boolean[Math.max(0, 2 * n - 1)];
    return placeRow(0, n, columns, down, up);
}

private static int placeRow(int row, int n, boolean[] columns,
                             boolean[] down, boolean[] up) {
    if (row == n) return 1;
    int count = 0;
    for (int col = 0; col < n; col++) {
        int d = row - col + n - 1;
        int u = row + col;
        if (columns[col] || down[d] || up[u]) continue;
        columns[col] = down[d] = up[u] = true;
        count += placeRow(row + 1, n, columns, down, up);
        columns[col] = down[d] = up[u] = false;
    }
    return count;
}
```

The occupancy arrays make validity checks $O(1)$, avoiding a scan of the partial board. Bit masks can compact the same sets and enumerate available columns efficiently, but the boolean version often communicates the invariant better.

Grid word search and path-local visited state

To find a word in a character grid, each recursive call represents a cell and word index. A cell may be reused by another independent start but not twice on the same active path. Therefore visited state is path-local and must be restored.

```
static boolean match(char[][] board, String word,
                    int row, int col, int index) {
    if (index == word.length()) return true;
    if (row < 0 || row >= board.length || col < 0
        || col >= board[row].length
        || board[row][col] != word.charAt(index)) return false;

    char saved = board[row][col];
    board[row][col] = '\0';
    boolean found = match(board, word, row + 1, col, index + 1)
        || match(board, word, row - 1, col, index + 1)
        || match(board, word, row, col + 1, index + 1)
        || match(board, word, row, col - 1, index + 1);
    board[row][col] = saved;
    return found;
}
```

Temporarily modifying the board saves a separate visited matrix, but it requires mutation permission and reliable restoration. The sample's `char` semantics are appropriate only if the board and word are explicitly UTF-16 code units or a limited alphabet. General user text requires a defined unit.

Short-circuiting still reaches restoration because the result is assigned before the board is repaired. Returning directly from one successful branch before restoring is a common bug.

Pruning must be proved safe

Pruning removes branches that cannot contain a valid or better solution. Useful forms include:

- feasibility pruning: remaining choices cannot satisfy a constraint;
- bound pruning: even the best possible completion cannot beat the current best;
- duplicate/symmetry pruning: equivalent branches would repeat results;
- ordering: explore promising candidates early so the best bound improves sooner;
- memoization: identical future subproblems need not be recomputed.

For a target sum with all remaining values nonnegative, a partial sum above the target can be pruned. If negative values are allowed, that rule is unsound because later values may reduce the sum. Every pruning condition depends on explicit assumptions.

```
partial score + optimistic maximum remaining <= current best
|
+-- branch cannot improve the answer
```

Branch-and-bound keeps a global incumbent best and computes an optimistic bound for each partial candidate. It may reduce practical search dramatically while retaining exponential worst-case complexity.

Memoization changes repeated search into dynamic programming

Backtracking paths sometimes reach the same future state. If the answer from that state does not depend on the full path history, cache it.

```

static boolean canSegment(String text, int index,
                          Set<String> dictionary, Boolean[] memo) {
    if (index == text.length()) return true;
    if (memo[index] != null) return memo[index];

    for (int end = index + 1; end <= text.length(); end++) {
        if (dictionary.contains(text.substring(index, end))
            && canSegment(text, end, dictionary, memo)) {
            return memo[index] = true;
        }
    }
    return memo[index] = false;
}

```

Here the future depends only on `index` because the dictionary and text are immutable inputs. If choices have limited counts, visited vertices, or remaining capacity, those belong in the memo key. An incomplete key returns answers computed under different constraints and silently corrupts correctness.

Memoization is most effective when many paths converge on a small state set. If each path has a unique visited mask, the cache may consume exponential memory without much reuse.

Recursion depth and iterative conversion

Recursive depth can be independent of total work. A balanced binary tree has $O(\log n)$ depth; a linked chain has $O(n)$. An input of one million nodes can overflow the Java stack even when the algorithm is linear.

An explicit stack stores frames on the heap and permits limits, cancellation, and resumability. For postorder behavior, a frame may need a phase or next-choice index:

```

record Frame(Node node, boolean expanded) {}

Deque<Frame> stack = new ArrayDeque<>();
stack.push(new Frame(root, false));
while (!stack.isEmpty()) {
    Frame frame = stack.pop();
    if (frame.node() == null) continue;
    if (frame.expanded()) {
        processAfterChildren(frame.node());
    } else {
        stack.push(new Frame(frame.node(), true));
        stack.push(new Frame(frame.node().right(), false));
        stack.push(new Frame(frame.node().left(), false));
    }
}

```

Converting recursion is not always "replace call with push." The explicit frame must preserve every piece of suspended local state and entry/exit timing.

Lazy generation and early termination

Materializing all solutions is often the wrong API. A search can accept a consumer and stop when the consumer returns false, or expose a custom iterator/spliterator with an explicit frame stack. This lets callers request the first solution, first k solutions, or process results incrementally.

Cancellation should be checked at safe intervals. In parallel search, tasks need a shared cancellation signal and a bounded split policy; spawning one task per branch can create more overhead than useful work. Work stealing can balance irregular branches, but shared best bounds and result ordering require synchronization.

Search order affects performance and reproducibility even when it does not affect the solution set. Trying constrained choices first can expose failure earlier; trying high-value choices first can improve a branch-and-bound incumbent. If tests, caches, or users expect deterministic output, sort candidates with a

stable total comparator and specify whether "first solution" depends on that order. Hash-set iteration is not a deterministic search policy.

Canonical output can eliminate duplicates when multiple paths represent the same logical solution. Normalize equivalent representations before recording or impose a construction order that generates only one representative. Deduplicating every completed result with a large hash set is often more expensive than avoiding symmetric branches.

Production applications

Backtracking appears in configuration solving, test-case generation, routing with constraints, policy evaluation, scheduling, query planning, and puzzle-like validation. Production solvers need time budgets, node-expansion limits, memory caps, cancellation, deterministic seeds, and diagnostic evidence explaining why no solution was found.

Record metrics such as expanded states, pruned states by reason, maximum depth, memo hits, incumbent improvements, elapsed time, and termination cause. A timeout result is not the same as "no solution." If approximate or best-so-far output is permitted, name the optimality guarantee.

Checkpointing a long search requires more than serializing the current path. The pending frontier, candidate ordering, incumbent, memo state, solver version, and input snapshot all influence continuation. An explicit frame stack is easier to checkpoint than the JVM call stack. For many services, bounded restart from an immutable input is simpler and safer than resumable search.

Untrusted inputs can intentionally trigger worst-case exponential work. Validate size and structure before search. Avoid logging full candidate paths when they contain secrets or produce enormous logs.

Testing recursive and backtracking code

Test empty input, one choice, impossible constraints, exactly one solution, many symmetric solutions, duplicates, maximum allowed depth, and early cancellation. Verify properties:

- every output satisfies all constraints;
- no valid solution is missing for small brute-force instances;
- no duplicate solution appears under the defined equality;
- mutable path state is restored after every branch;
- pruning produces the same result as unpruned search on small inputs;
- memoized and nonmemoized versions agree;
- time or expansion limits report incomplete search honestly.

Instrument path snapshots in tests to catch state leakage. Randomized small cases plus an intentionally slow oracle are extremely effective.

Common senior-level traps

- Writing recursion without a measurable progress argument.
- Claiming tail recursion is stack-safe in Java.
- Recording the same mutable path object instead of copying it.
- Forgetting undo after a successful or exceptional branch.
- Using global request state that leaks across calls or threads.
- Treating exponential output as a linear-time algorithm.
- Applying a pruning rule whose assumptions do not hold.

- Memoizing with an incomplete state key.
- Marking path-local visited state globally.
- Using recursion on attacker-controlled depth.
- Materializing all results when the caller needs only one or a bounded prefix.

A strong interview answer

I define the recursive contract, base case, and a progress measure before writing code. For backtracking, the state represents one partial candidate; each branch applies a choice, explores it, and restores exactly the previous state. I estimate the search tree and output size, then add only pruning rules I can prove safe under the input assumptions. If different paths reach the same future state, I memoize a complete state key. In Java I consider stack depth, copy versus mutate-and-undo, cancellation, and bounded/lazy output. For production search I distinguish no solution from resource-limit termination and expose expansion and pruning metrics.

Chapter summary

Recursion turns self-similar structure into nested calls; backtracking turns choices into a depth-first decision tree. Correctness depends on explicit contracts, progress, state ownership, and restoration. Pruning uses feasibility, bounds, symmetry, or repeated-state knowledge to reduce exploration without removing valid answers. Worst-case exponential growth still matters, so production designs need limits, lazy results, cancellation, observability, and honest termination semantics.

Revision Notes

- Recursion needs a state, base case, and measurable progress.
- Java does not guarantee tail-call elimination.
- Stack space is proportional to maximum active depth.
- Backtracking follows choose, apply, explore, and undo.
- Copy mutable paths when recording results.
- A combination start index prevents reordered duplicates.
- Distinct permutations produce $n!$ outputs; subsets produce 2^n outputs.
- Duplicate permutation pruning removes symmetric sibling choices.
- Path-local visited state must be restored; global visited has different meaning.
- Occupancy sets can make constraint checks $O(1)$.
- Pruning is correct only when its assumptions are proved.
- Branch-and-bound uses an optimistic bound and incumbent best.
- Memoization keys must contain every state component affecting the future.
- Explicit stacks avoid call-stack overflow and can preserve entry/exit phases.
- Lazy generation avoids collecting all solutions.
- Production search needs time, depth, expansion, output, and memory limits.
- A timeout means incomplete search, not proof that no solution exists.

Likely interview question: Recursion versus backtracking?

Recursion is a control technique in which a function solves smaller instances. Backtracking is a search strategy usually implemented recursively: it makes a choice, explores, and undoes it to try siblings. A recursive algorithm need not backtrack.

Likely interview question: How do you prove a pruning rule is safe?

State the invariant and show that every completion below the pruned node is infeasible, duplicate, or cannot beat the incumbent. The proof must use valid domain assumptions; for example, sum-based pruning changes when negative values are allowed.

Likely interview question: When should you memoize backtracking?

Memoize when many paths reach the same future subproblem and its answer depends only on a compact state. Include every relevant constraint in the key. Avoid a cache when states are mostly unique or memory would become the limiting resource.

Likely interview question: How do you make recursive search production-safe?

Validate input, bound depth and expansions, support cancellation and deadlines, stream bounded results, avoid shared mutable request state, expose termination reason and search metrics, and use an explicit stack when depth is not safely bounded.